

Binary Search Tree - II

Querying a BST : finding min, max, predecessor, successor, or search

Search : returns a pointer to a node with key k if one exists in the subtree, else returns NIL.

Tree-Search(x, k) → root of a subtree in T
→ the value to search for

if $x == \text{NIL}$ or $k == x.\text{key}$
return x

if $k < x.\text{key}$

return Tree-Search($x.\text{left}, k$)

else

return Tree-Search($x.\text{right}, k$)

Running time of Tree-Search is $O(h)$

where h is the height of the tree

Iterative -Tree-Search (x, k)

While $x \neq \text{NIL}$ and $k \neq x.\text{key}$

 if $k < x.\text{key}$

$x = x.\text{left}$

 else

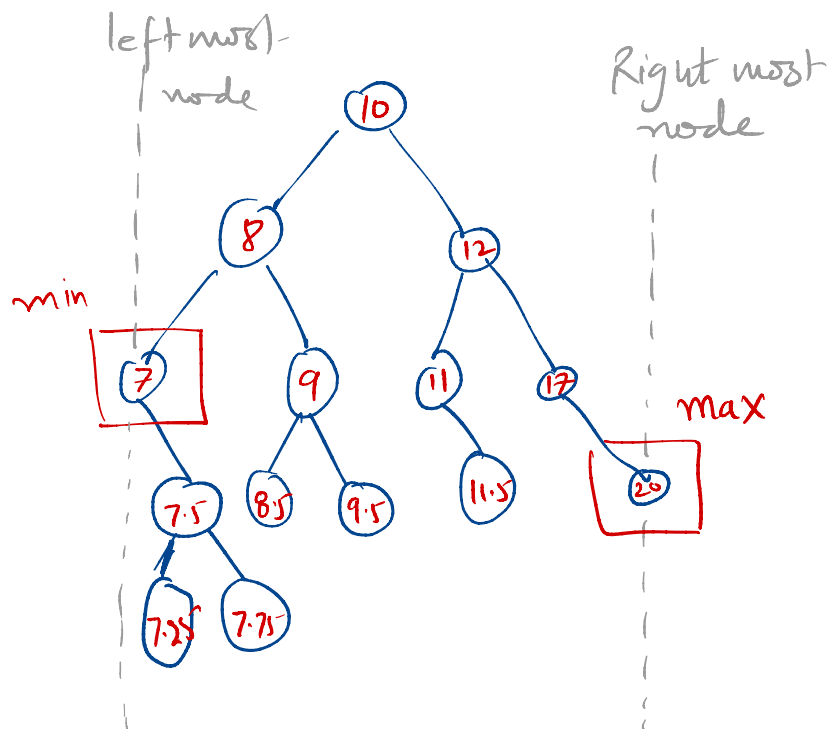
$x = x.\text{right}$

return x

$\rightarrow O(h)$

Minimum / maximum

$\rightarrow$ returns a pointer to min/max element in a subtree rooted at node x (assumed to be not NIL)



Tree - Minimum (x)

```
while x.left ≠ NIL
    |   x = x.left
return x
```

Tree - Maximum (x)

```
while x.right ≠ NIL
    |   x = x.right
return x
```

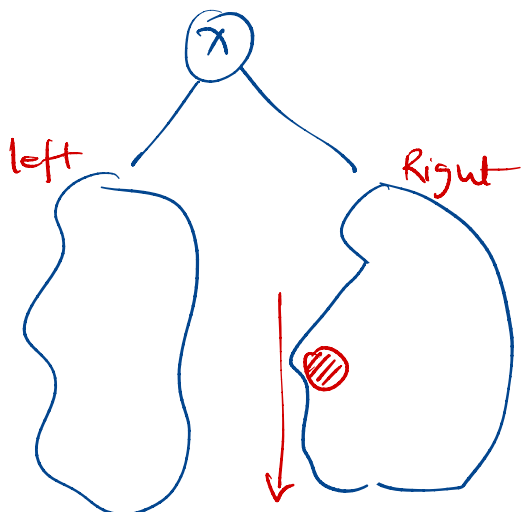
- Successor and predecessor

→ Find Successor of a given node in the sorted order determined by the inorder walk.

⇓

Define Successor as the next node visited in an inorder tree walk.

Two cases : (I) If the right subtree of x is non empty



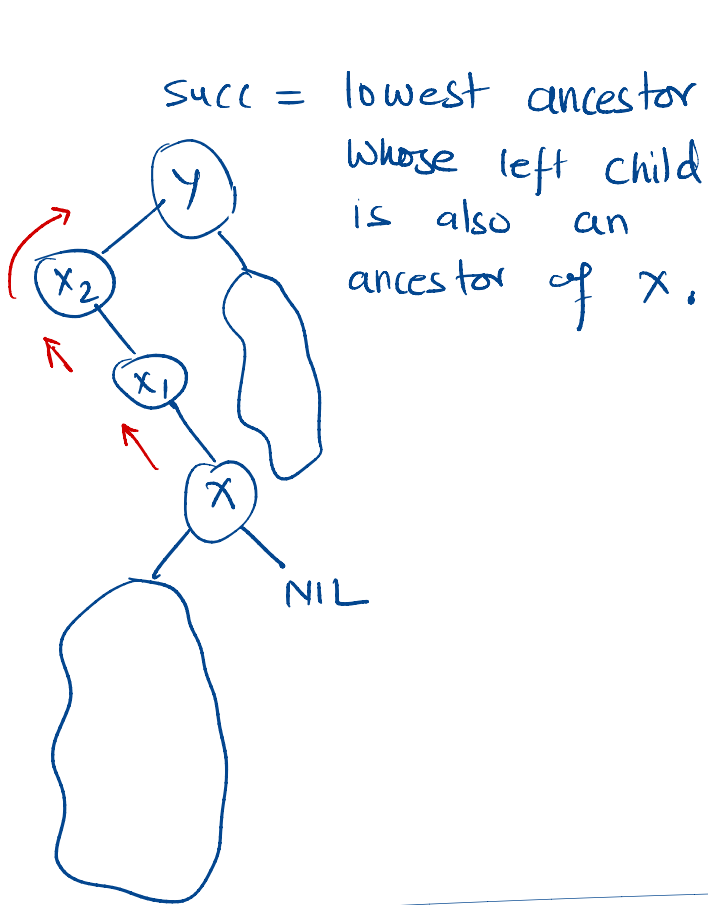
Succ = leftmost node

→ Successor = leftmost node in x's right subtree

↓
Call

Tree-Minimum (x.right)

(II) If the right subtree of x is empty



Then y is a successor, if y is the lowest ancestor of x whose left child is also an ancestor of x .

Tree - Successor (x)

if $x.\text{right} \neq \text{NIL}$

return Tree-Minimum($x.\text{right}$)

else

$y = x.p$

While $y \neq \text{NIL}$ and $x == y.\text{right}$

$x = y$

$y = y.p$

return y
